Overlapping Ring Monitoring Algorithm in TIPC

Jon Maloy, Ericsson Canada Inc. Montreal

April 7th 2017

PURPOSE

When a cluster node becomes unresponsive due to crash, reboot or lost connectivity we want to:

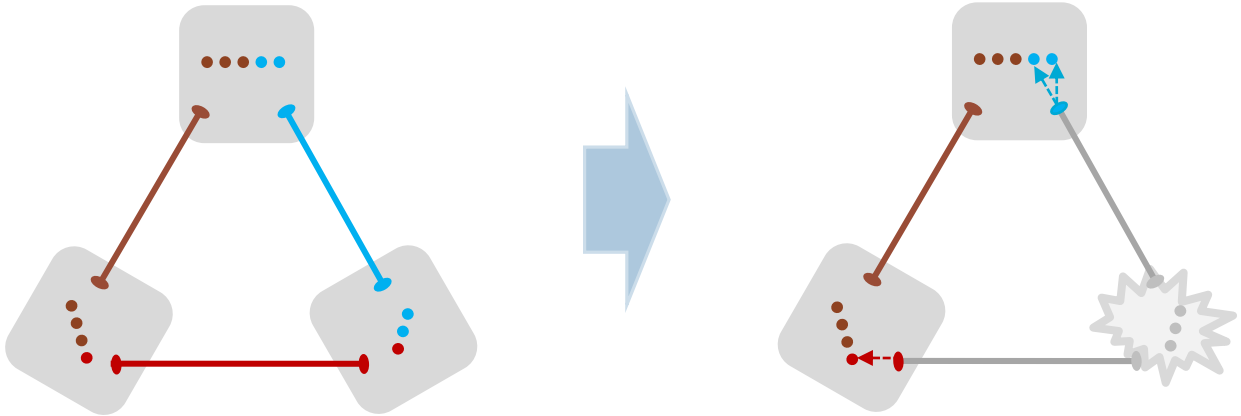
- Have all affected connections on the remaining nodes aborted
- Inform other users who have subscribed for cluster connectivity events
- Within a well-defined short interval from the occurrence of the event

COMMON SOLUTIONS

1) Crank up the connection keepalive timer
   - Network and CPU load quickly gets out of hand when there are thousands of connections
   - Does not provide a neighbor monitoring service that can be used by others
2) Dedicated full-mesh framework of per-node daemons with frequently probed connections
   - Even here monitoring traffic becomes overwhelming when cluster size > 100 nodes
   - Does not automatically abort any other connections

TIPC SOLUTION: HIERARCHY + FULL MESH

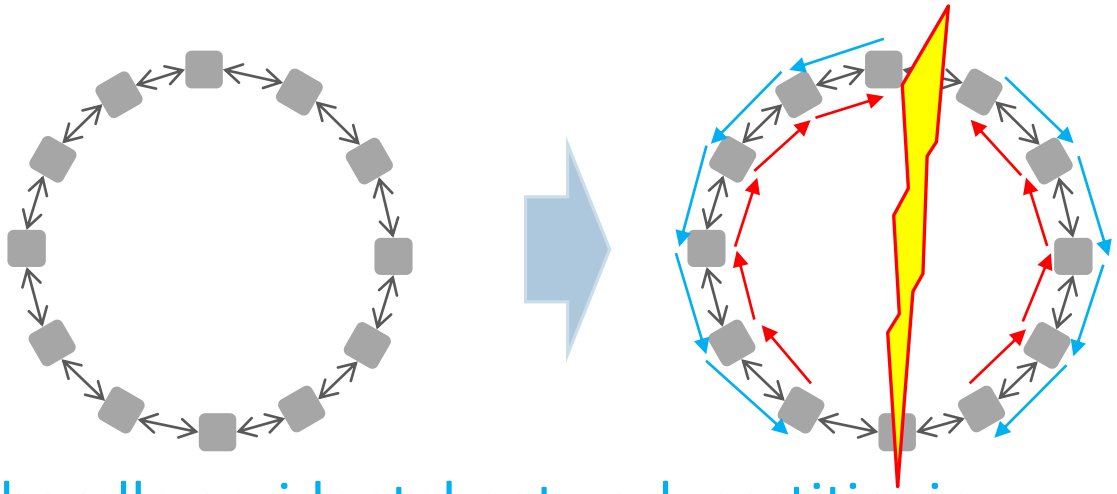
- Full-mesh framework of frequently probed node-to-node “links”
  - At kernel level
  - Provides generic neighbor monitoring service
- Each link endpoint keeps track of all connections to peer node
  - Issues “ABORT” message to its local socket endpoints when connectivity to peer node is lost



- Even this solution causes excessive traffic beyond ~100 nodes
  - CPU load grows with ~N
  - Network load grows with ~N*(N-1)

OTHER SOLUTION: RING

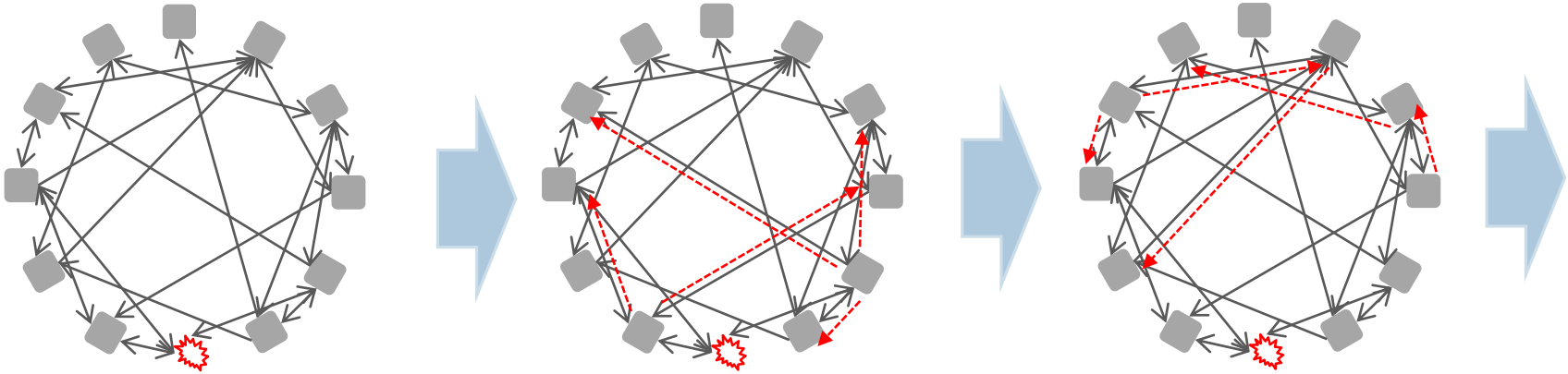
- Each node monitors its two nearest neighbors by heatbeats
  - Low monitoring network overhead, - increases by ~2*N
- Node loss can also be detected through loss of an iterating token
  - Both solutions offered by Corosync



- Hard to handle accidental network partitioning
  - How do we detect loss of nodes not adjacent to fracture point in opposite partition?
  - Consensus on ring topology required

OTHER SOLUTION: GOSSIP PROTOCOL

- Each node periodically transmits its known network view to a randomly selected set of known neighbors
  - Each node knows and monitors only a subset of all nodes
  - Scales extremely well
  - Used by BitTorrent client Tribler



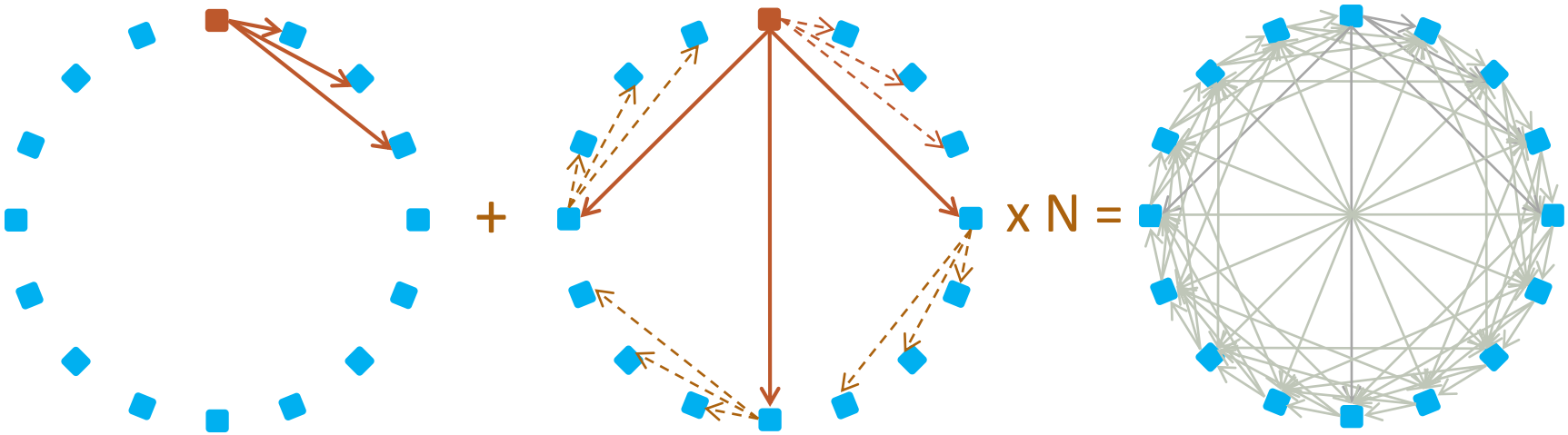
- Non-deterministic delay until all cluster nodes are informed
  - Potentially very long because of the periodic and random nature of event propagation
  - Unpredictable number of generations to reach last node
  - Extra network overhead because of duplicate information spreading

THE CHALLENGE

Finding an algorithm which:

- Has the scalability of Gossip, but with
  - A deterministic set of peer nodes to monitor and update from each node
  - A predictable number of propagation generations before all nodes are reached
  - Predictable, well-defined and short event propagation delay
- Has the light-weight properties of ring monitoring, but
  - Is able to handle accidental network partitioning
- Has the full-mesh link connectivity of TIPC, but
  - Does not require full-mesh active monitoring

THE ANSWER:
OVERLAPPING RING MONITORING



(√N – 1) Local Domain Destinations

(√N – 1) Remote "Head" Destinations

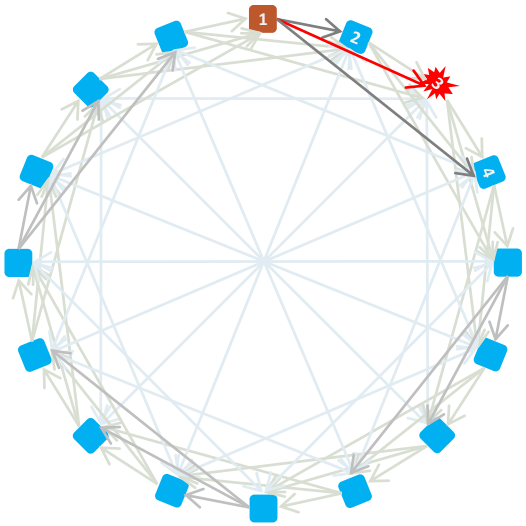
2 x N x (√N – 1) Actively Monitored Links

- Sort all cluster nodes into a circular list
  - All nodes use same algorithm and criteria
- Select next ⌊√N⌋ - 1 downstream nodes in the list as "local domain" to be actively monitored
  - CPU load increases by ~√N
- Distribute a record describing the local domain to all other nodes in the cluster

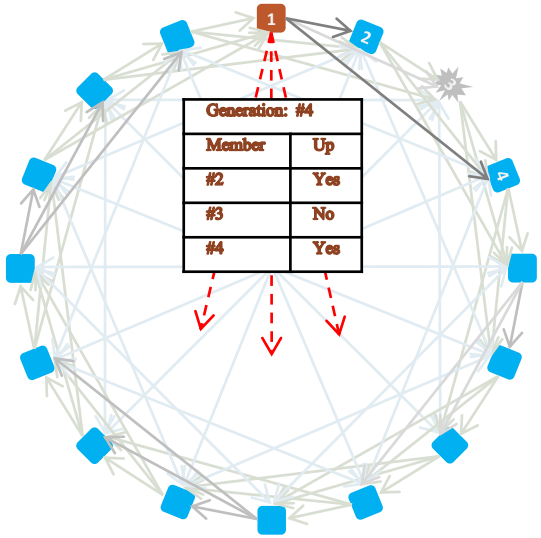
- Select and monitor a set of "head" nodes outside the local domain so that no node is more than two active monitoring hops away
  - There will be ⌈√N⌉ - 1 such nodes
  - Guarantees failure discovery even at accidental network partitioning
- Each node now monitors 2 x (√N – 1) neighbors
  - 6 neighbors in a 16 node cluster
  - 56 neighbors in an 800 node cluster

- All nodes use this algorithm
- In total 2 x (√N - 1) x N actively monitored links
  - 96 links in a 16 node cluster
  - 44,800 links in an 800 node cluster

LOSS OF LOCAL DOMAIN NODE



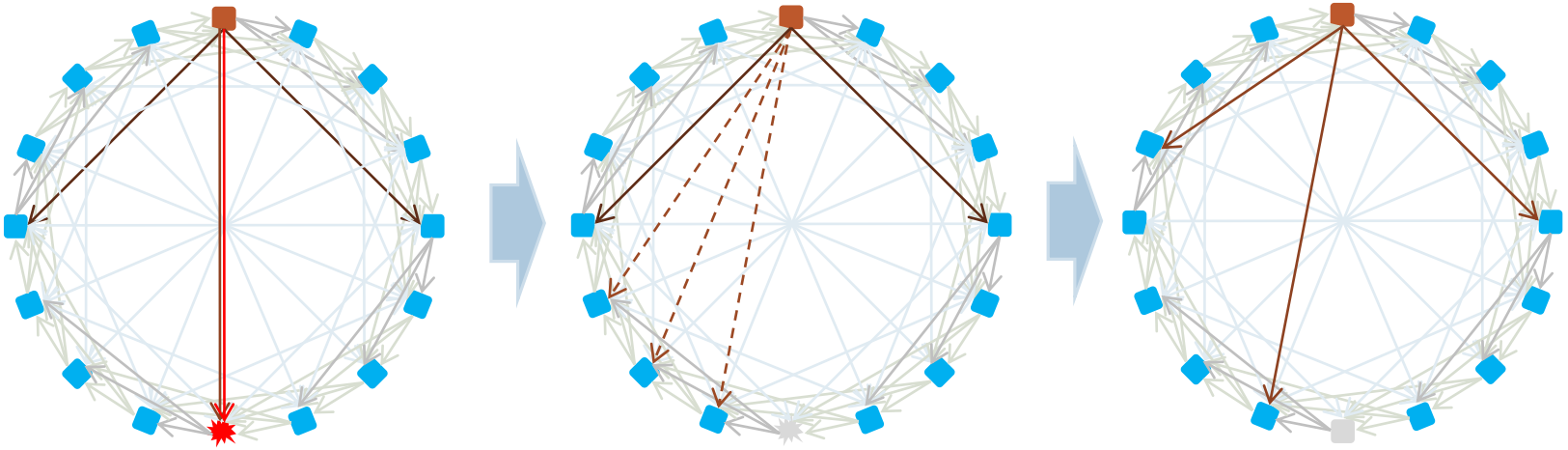
State change of local domain node detected



Domain record distributed to all other nodes in cluster

- A domain record is sent to all other nodes in cluster when any state change (discovery, loss, re-establish) is detected in a local domain node
- The record keeps a generation id, so the receiver can know if it really contains a change before it starts parsing and applying it
- It is piggy-backed on regular unicast link state/probe messages, which must always be sent out after a domain state change
- May be sent several times until the receiver acknowledges reception of the current generation
- Because probing is driven by a background timer, it may take up to 375 ms (configurable) until all nodes are updated

LOSS OF ACTIVELY MONITORED HEAD NODE



Node failure detected

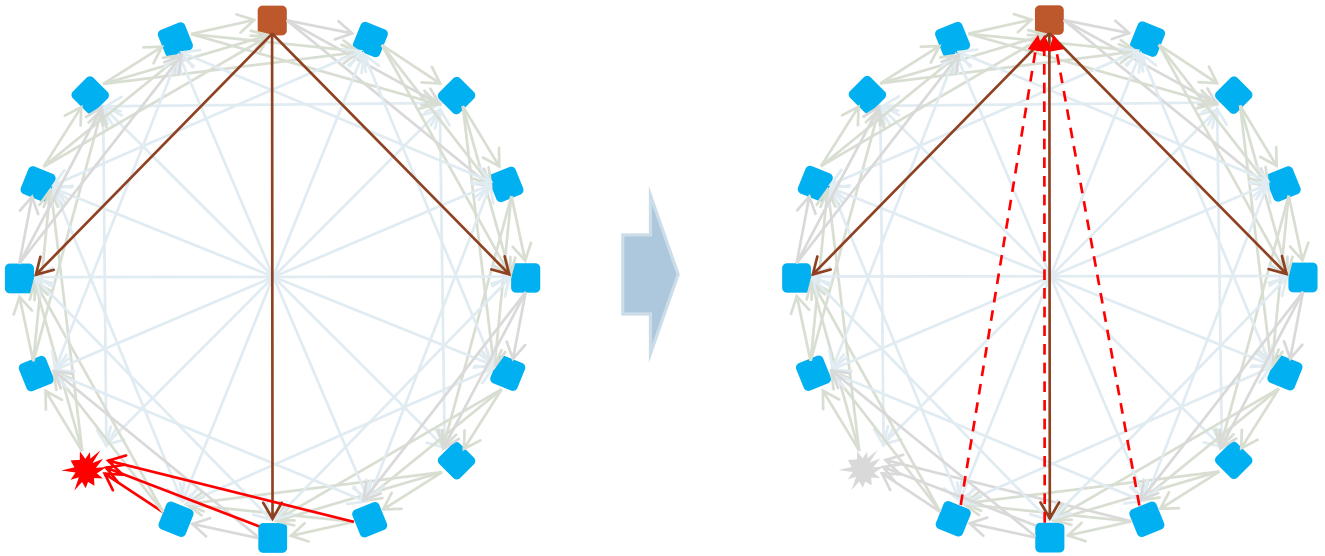
Brief confirmation probing of lost node's domain members

After recalculation

- The two-hop criteria plus confirmation probing eliminates the network partitioning problem
- If we really have a partition worst-case failure detection time will be
  - $T_{failmax}$ = *2 x active failure detection time*
- Active failure detection time is configurable
  - 50 ms – 10 s
  - Default 1.5 s in TIPC/Linux 4.7

← Actively monitored nodes outside local domain

LOSS OF INDIRECTLY MONITORED NODE



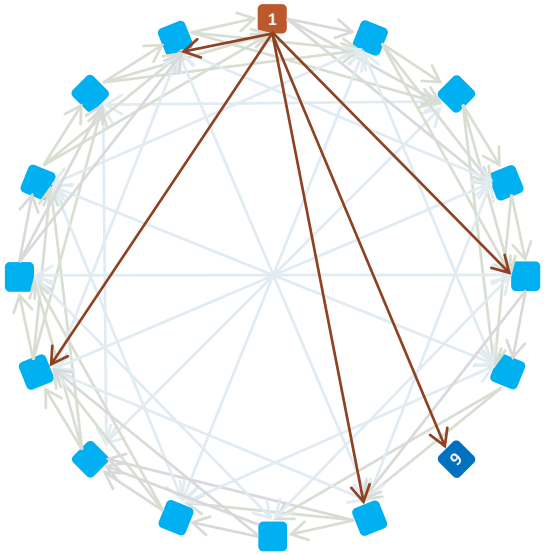
Actively monitoring neighbors discover failure

Actively monitoring neighbors report failure

- Max one event propagation hop
- Near uniform failure detection time across the whole cluster
  - $T_{failmax}$ = *active failure detection time + (1 x event propagation hop time)*

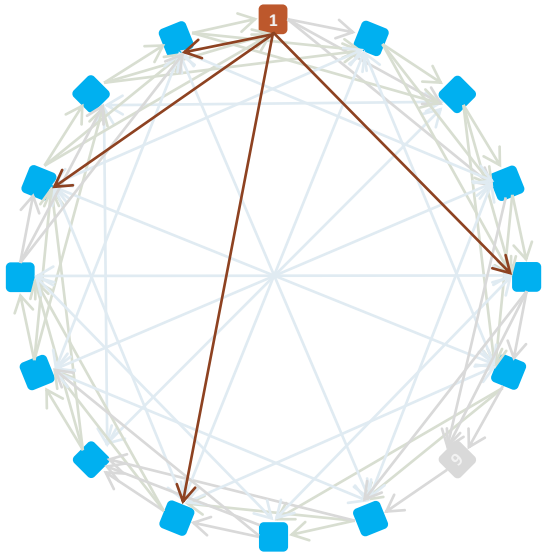
← Actively monitored nodes outside local domain

DIFFERING NETWORK VIEWS



A node has discovered a peer that nobody else is monitoring

- Actively monitor that node
- Add it to its circular list according to algorithm (as local domain member or “head”)
- Handle its domain members according to algorithm (“applied” or “non-applied”)
- Continue calculating the monitoring view from the next peer



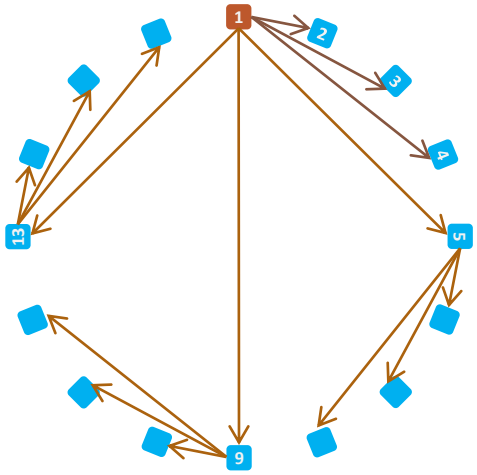
A node is unable to discover a peer that others are monitoring

- Don’t add the peer to the circular list
- Ignore it during the calculation of the monitoring view
- Keep it as “non-applied” in the copies of received domain records
- Apply it to the monitoring view if it is discovered at a later moment

Transiently, this happens all the time, and must be considered a normal situation

← Actively monitored nodes outside local domain

STATUS LISTING OF 16 NODE CLUSTER



File Edit View Bookmarks Settings Help

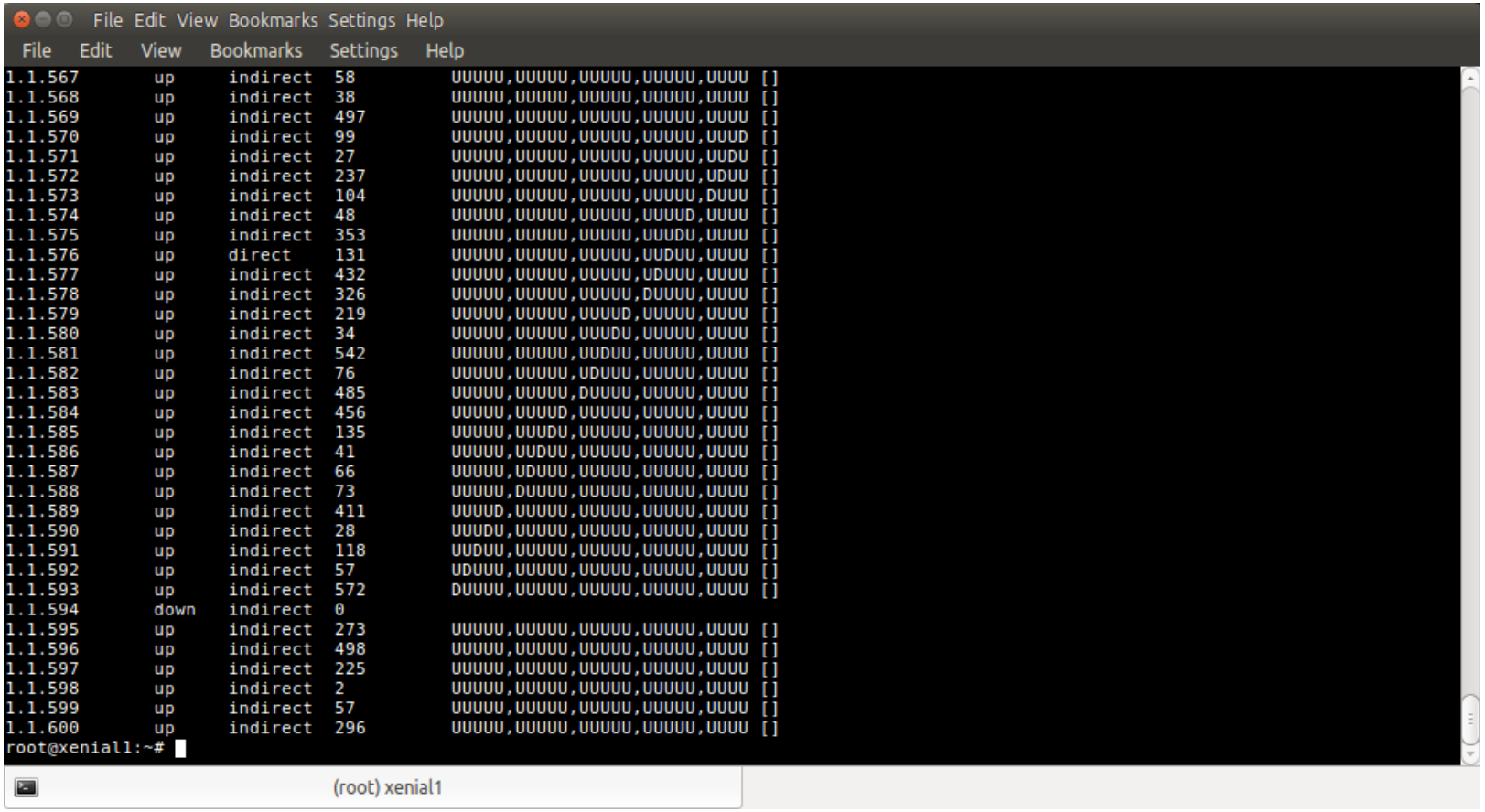
File Edit View Bookmarks Settings Help

```
root@xenial1:~#
root@xenial1:~# tipc node get addr
<1.1.1>
root@xenial1:~# tipc link mon list

bearer eth:tap1
node          status monitored generation applied_node_status [non_applied_node:status]
1.1.1         up     direct    2          UUU []
1.1.2         up     direct    3          UUU []
1.1.3         up     direct    5          UUU []
1.1.4         up     direct    6          UUU []
1.1.5         up     direct    7          UUU []
1.1.6         up     indirect  8          UUU []
1.1.7         up     indirect  9          UUU []
1.1.8         up     indirect  10         UUU []
1.1.9         up     direct    11         UUU []
1.1.10        up     indirect  12         UUU []
1.1.11        up     indirect  13         UUU []
1.1.12        up     indirect  14         UUU []
1.1.13        up     direct    15         UUU []
1.1.14        up     indirect  15         UUU []
1.1.15        up     indirect  15         UUU []
1.1.16        up     indirect  15         UUU []
root@xenial1:~#
```

(root) xenial1

STATUS LISTING OF 600 NODE CLUSTER



THE END